

Ablation Study of a From-Scratch Encoder-Decoder Transformer

Surya Manoj Pentakota

manoj.pentakota@research.iiit.ac.in

Abstract

This report presents an ablation study of an encoder-decoder Transformer model trained on dataset of 5000 pairs of encrypted binary sentences to plain text mappings. We study on different configurations of the model with single architectural choice difference in each. Starting with base model C1 (multi-head attention, LayerNorm, sinusoidal positional encoding, byte-level BPE), C2 uses RoPE for positional encoding, C3 uses grouped-query attention, C4 RMSNorm, C5 replaces subword tokenization with token free byte latent transformer. All the configurations share the same hyperparameters, training recipe and evaluation protocols (greedy decoding with bit-accuracy, sentence-accuracy, Levenshtein distance, and BLEU/ROUGE for tokenized models)

Our results show that the token-free configuration (C5) reaches the best accuracy (99.96% bit, 96% sequence accuracy) at the same peak memory, with tradeoff being it is +3.36M parameters bigger and +55% per-step time at training. Among the tokenized models, LayerNorm/rmsnorm and sinusoidal/RoPE choices are comparable except for RoPE, which fails to learn the code-text mapping (55.2% bit accuracy), while grouped-query attention (91.2%) underperforms the base by about 5%. The study isolates each component choice in the architecture and quantifies their performances by accuracy and efficiency of each choice.

1 Introduction

The Transformer [1] has become a default for sequence understanding in MLP, but its performance heavily relies on the architectural choices of its components which all carry their pros and cons at different task definitions. So for our study we isolate each on the component choice and perform a controlled ablation study on a well defined task: deciphering given bit sequence to plain text. As cipher is a bit sequence mapping to plain text the token is token mapping is not direct and hence we use repeating-XOR of period 8 (1 byte of cipher text per plaintext character) making this a position-dependent cipher byte to text character mapping (easier task definition then learning full sequence to sequence at once).

We train five configurations of an encoder-decoder Transformer (C1-C5). Every configuration differs from the base model (C1) in exactly one component choice, so any difference in test accuracy or training cost can be attributed to that component alone. The configurations cover positional encoding (RoPE, C2), grouped-query attention (C3), RMSNorm (C4) and a token-free Byte Latent Transformer [3] (C5). We report accuracy metrics (bit and sequence accuracy, Levenshtein

distance, BLEU, ROUGE) together with a compute cost (parameters, peak memory, steps per second), and we will focus more on the tradeoff between memory and accuracy introduced by the token-free BLT approach.

2 Data and Task Formulation

The corpus consists of 5,000 aligned (ciphertext, plaintext) pairs. The plaintext is ASCII text; each character is one byte, and the ciphertext is obtained by XOR-ing the byte stream with a repeating 8-byte key, making the mapping is position-local where the cipher byte at position p depends only on the plaintext byte at p , so no long-range context is required. Because a full sentence can be up to 21,360 cipher bits, we train on phase-aligned *windows* of 32 plaintext characters where every window is a multiple of the key period, so each window starts at key phase 0 and inherits the identical position-local mapping. Corpus is split 80/10/10 line-wise before windowing (a line never appears in two splits), yielding 20,000 training windows (capped sample) and 2,000 validation and 2,000 test windows. Configurations with tokenizer use byte-level BPE of vocab size 1,024 on both sides; the token-free configuration (C5) works on packed byte values directly.

3 Model Implementation

3.1 Encoder–Decoder Transformer

We implement a pre-norm encoder-decoder Transformer. Each encoder block applies self-attention followed by a position-wise FFN with GELU, each preceded by LayerNorm/RMSNorm; each decoder block adds a cross-attention layer whose memory keys are normalized separately. Attention projections are bias-free, and the FFN applies dropout before and after the activation. Embeddings are scaled by $\sqrt{d_{\text{model}}}$, the decoder output head shares the target embedding matrix, and parameters are initialized as $\mathcal{N}(0, 0.02)$ (embeddings) / Xavier (linears), with the padding-id row zeroed. Label smoothing ($\epsilon = 0.1$) is applied to the training loss only. The base configuration uses multi-head attention (8 heads, $d_k = 32$), LayerNorm and sinusoidal positional encoding.

3.2 From-Scratch Byte-Level BPE

Both vocabularies are learned with a byte-level BPE [2] implemented from scratch: the alphabets are 256 byte values plus three specials; merges are learned by repeatedly combining the most frequent adjacent pair up to a size of 1,024; tie-breaking is deterministic and decoding is lossless.

3.3 Byte Latent Transformer (C5)

C5 replaces the subword interface with a Byte Latent Transformer [3] pipeline: (i) every 8 cipher bits are packed into one byte value, so the model reads the data rather than its ASCII encoding; (ii) a 1-layer, 4-head local encoder with a 16-byte sliding window and sinusoidal byte positions produces byte states, pooled into fixed 4-byte patch representations by cross-attention with a learned query; (iii) the same 3-layer global model runs at patch resolution with an input shift by one patch

(learned start patch) so patch j conditions only on patches $< j$; (iv) a causal byte-level local decoder conditions on three pathways: its own patch state (gathered per byte), the patch history (cross-attention masked to seen patches), and the source byte states (cross-attention), the last being important and irreplaceable for exact reconstruction. Training is teacher-forced next-byte prediction ($\text{tgt}[: -1]$ vs. $\text{tgt}[1 :]$) and decoding is byte-autoregressive, matching the training layout exactly.

4 Experimental Setup

4.1 Configurations

Table 1: Architecture and tokenization of the five configurations. All models share the base hyperparameters: $d_{\text{model}} = 256$, 8 heads, $d_{\text{ff}} = 1024$, 3 encoder / 3 decoder layers, batch 64, 32 epochs.

Config	Attention	Norm	Positional	Cipher tokens	Plain tokens
C1	MHA	LayerNorm	Sinusoidal	BPE (1024)	BPE (1024)
C2	MHA	LayerNorm	RoPE	BPE (1024)	BPE (1024)
C3	GQA (2 KV)	LayerNorm	Sinusoidal	BPE (1024)	BPE (1024)
C4	MHA	RMSNorm	Sinusoidal	BPE (1024)	BPE (1024)
C5	MHA	LayerNorm	Sinusoidal	bytes (260)	bytes (260)

Table 2: Compute profile of the five configurations (T4 GPU). Parameter counts include the tied output head; K/V columns are the key/value projection weights. Peak memory is the measured on-run peak (`torch.cuda.max_memory_allocated`, process-wide; all configs run at the same steady-state ~ 497 MB). Time is the total wandb-measured wall time (training + per-epoch validation + final test evaluation).

Config	Params	K/V params	Peak mem (MB)	s/step	Time (min)
C1	6,048,256	1,179,648	497	0.042	16.7
C2	6,048,256	1,179,648	497	0.042	16.7
C3	5,163,520	294,912	497	0.044	17.0
C4	6,043,136	1,179,648	497	0.040	16.4
C5	9,407,236	1,966,080	497	0.065	12.1

Note: the peak-memory figures are the PyTorch allocator peak (`torch.cuda.max_memory_allocated`), measured on the running process. They are lower bounds and do not correspond to the actual device-side GPU memory usage, which the GPU monitor reports at ~ 4.6 – 7.0 GB (including the CUDA context and the caching allocator). All configs share the same steady state, so the relative ordering is best read from the parameter/speed columns rather than from this row.

4.2 Training Details

All configurations share an identical training recipe with 32 epochs of batch 64 ($\approx 10,000$ steps over the 20,000 training windows) this is implemented rather than early stopping to give the same

training conditions and observe the convergence patterns, AdamW with $\beta = (0.9, 0.98)$, weight decay 0.01, gradient clipping at 1.0, base learning rate 8×10^{-4} with a 500-step linear warmup followed by a cosine decay to 10% of the peak, and fp16 mixed precision with gradient scaling. The model with the best validation loss (plain cross-entropy) is checkpointed during training and its weights are used for the final test evaluation; the test set is never used for any selection decision with one seed (42) used throughout all runs.

4.3 Evaluation

All reported numbers use greedy decoding. Tokenized configurations (C1-C4) decode until an EOS token; the BLT (C5) decodes bytes autoregressively until end-of-sequence. We report, on the 2,000 window test set: *bit accuracy* (fraction of exact bit matches, with extra bytes of the longer string counted as mismatches), *sequence accuracy* (fraction of exactly reconstructed windows), *Levenshtein distance* (average per-window edit distance in characters), and for the tokenized models additionally *BLEU* (corpus-level, smoothed) and *ROUGE*–1/2/*L* (F-measure). BLEU and ROUGE are n-gram metrics and therefore not applicable to the byte-level output of C5.

5 Results

Test-set results are summarized in Table 3 and the compute profile in Table 2; training/validation loss curves are shown in Figure 1. The base model (C1) already performs well (96.68% bit accuracy, 65.6% exact sequences), which confirms that the position-local mapping is highly learnable; the per-configuration analyses below focus on what each single-component change did to that well-performing base.

Table 3: Test-set results with greedy decoding (best-val checkpoint). BLEU/ROUGE are reported for the tokenized models only (C1-C4); C5 is byte-level and thus not applicable.

Config	Bit acc %	Seq acc %	Lev	BLEU	ROUGE-1	ROUGE-2	ROUGE-L
C1	96.68	65.60	0.68	0.9688	0.9282	0.8775	0.9282
C2	55.20	0.00	31.34	0.2849	0.2991	0.0641	0.2732
C3	91.20	38.70	2.15	0.8967	0.8354	0.7123	0.8354
C4	96.44	63.00	0.71	0.9663	0.9219	0.8666	0.9219
C5	99.96	96.00	0.05	–	–	–	–

5.1 Per-Configuration Analysis

5.1.1 C1 (base)

The base model achieves 96.68% bit accuracy and 65.6% exact sequences (BLEU 0.969, ROUGE-1 0.928). Errors are small, the average Levenshtein distance is 0.68 characters per window, i.e. nearly all content is reconstructed correctly with only a few character-level details wrong.

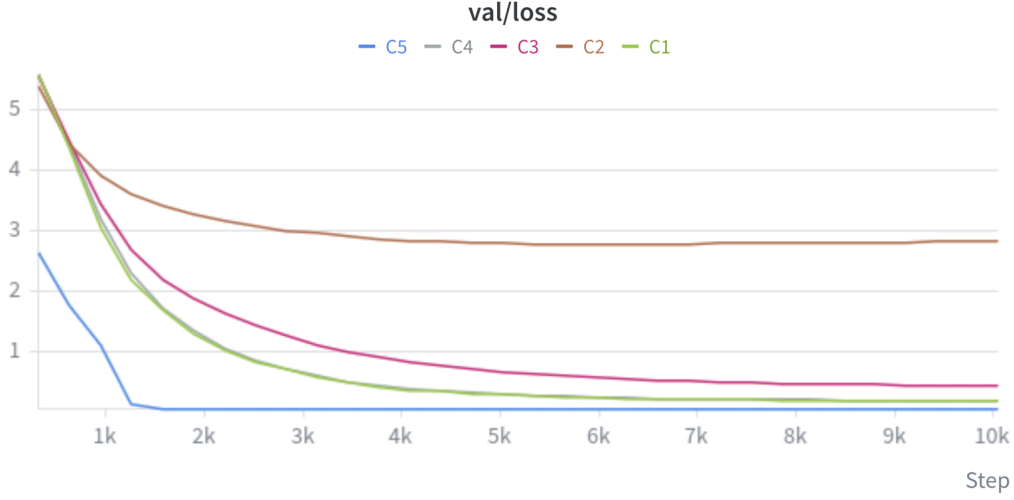


Figure 1: Validation loss (per epoch) for the five configurations: C1/C3/C4 descend to ~ 0.2 – 0.7 , C5 stays near 0.1, and C2 (RoPE) plateaus at ~ 2.8 – visible even where the tables cannot show training dynamics.

5.1.2 C2 (RoPE)

Replacing sinusoidal positions with rotary position embeddings [4] degrades trainings, validation loss plateaus near 2.8 while training loss keeps decreasing, and test accuracy collapses to 55.2% bit / 0% exact sequences (lev 31.3). This may be due to a relative-only position encoding cannot anchoring the absolute position phase of a phase-aligned, position-local cipher; we verified ours is the canonical rotation, so the failure must be the interaction.

5.1.3 C3 (Grouped-Query Attention)

Sharing KV across 2 heads (grouped-query attention [6], vs. 8) costs accuracy: 91.20% bit / 38.70% sequence vs. 96.68% / 65.60%, while removing 884,736 KV weights (294,912 vs. 1,179,648). Per-head KV fidelity matters more than the parameter savings on this reconstruction task; GQA’s benefit applies when KV-cache memory is the binding constraint.

5.1.4 C4 (RMSNorm)

RMSNorm [5] matches LayerNorm within noise (96.44% bit, 63.00% sequence, lev 0.71 vs. 0.68) while removing learned biases and the mean-centering step, and is marginally faster (0.040 vs. 0.042 s/step) a clean, cost-free normalization choice.

5.1.5 C5 (Byte Latent Transformer)

The token-free model is the best: 99.96% bit, 96.00% exact sequences, average Levenshtein 0.05 characters, essentially exact reconstruction at +3.36M parameters and +55% s/step (peak memory

unchanged, 497 MB). Because the cipher is a per-byte position-local mapping, the BLT’s byte-level local modules and patch-level global transformer track it directly, without the information bottleneck of a learned vocabulary. BLT performed extremely well on this task as the source sequence has near 0 entropy as it is bit sequence but on text with high entropy the results may vary for BLT but still a variable option with its gains outweighing the trade-offs.

6 Discussion

Component deltas. RoPE (C2) is the only change that hurt substantially – 55.2% vs. 96.68% bit accuracy, with a validation plateau while the training loss kept falling, the relative only encoding cannot anchor the absolute position phase the task needs, and the model overfits the training windows. GQA (C3) trades accuracy (91.20%) for 884,736 fewer KV weights, a poor deal for exact reconstruction, but valuable when KV-cache memory is the constraint. RMSNorm (C4) is accuracy-neutral (96.44%) and slightly faster, with fewer learned parameters, a free drop-in. BLT (C5) is the biggest gain and the biggest cost: 99.96% vs. 96.68% bit accuracy for +3.36M parameters and +55% s/step at unchanged 497 MB peak memory.

Memory-accuracy tradeoffs of the token-free approach. The BLT replaces the learned-vocabulary interface with byte-level local modules at 16-byte windows, concentrating compute in a global patch-level transformer. On this task the tradeoff is favorable, byte-level exactness (99.96% bit, 0.05 edit distance) at modest compute cost, because the cipher is a per-byte bijection matching the windowed byte design, and because the local decoder’s source-byte pathway frees the global model from compressing every byte into patch representations, the element most responsible for exact reconstruction. On tasks with long-range context that must be compressed through patch representations, the extra local compute would instead buy less accuracy per step than a subword tokenization implementation at the same budget.

7 Conclusion

We presented a controlled ablation of a from-scratch encoder-decoder Transformer on a position-local cipher task, changing exactly one component per configuration. The token-free Byte Latent Transformer (C5) delivered the best results (99.96% bit, 96.00% sequence accuracy) at unchanged 497 MB peak memory, with +3.36M parameters and +55% per-step cost, a favorable tradeoff for this byte-local task. Among the tokenized configurations, RMSNorm (C4) matched the base at lower cost, GQA (C3) traded accuracy for KV savings, and RoPE (C2) failed on the phase-coded data, which we attribute to the missing absolute position anchor. The study shows that single-component changes can be decisive on structured tasks, and that the token-free BLT principle local byte models meeting a global patch transformer is a strong and cost-effective match when the underlying mapping is byte-local.

8 Result Artifacts

Wandb: <https://wandb.ai/suryamanojphy31-iiit-hyderabad/anlp-assignment-1/workspace>

Huggingface: <https://huggingface.co/Surya4/anlp-assignment-1>

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, et al. “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, 2017.
- [2] R. Sennrich, B. Haddow, and A. Birch. “Neural Machine Translation of Rare Words with Subword Units,” *Proc. of the 54th Annual Meeting of the ACL*, 2016.
- [3] A. Artetxe, et al. “Byte Latent Transformer: Patches Scale Better Than Tokens,” arXiv:2412.09871, 2024.
- [4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. “RoFormer: Enhanced Transformer with Rotary Position Embedding,” arXiv:2104.09864, 2021.
- [5] B. Zhang and R. Sennrich. “Root Mean Square Layer Normalization,” *Advances in Neural Information Processing Systems*, 2019.
- [6] J. Ainslie, J. Lee-Thorp, M. de Jong, et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” *EMNLP*, 2023.